

Intermediate to Advanced Python Data Types Interview Questions and Answers

1. Difference Between `is` and `==`

`==`

Compares values

Equality operator

Example:

```
a = [1,2]
b = [1,2]
print(a == b)
print(a is b)
```

`is`

Compares memory location

Identity operator

Output:

True
False

2. Why Small Integers are Cached in Python?

Python caches small integers (-5 to 256) for memory optimization and performance.

```
a = 100
b = 100
print(a is b)
```

Output:

True

3. Explain Mutable Default Argument Problem

```
def add_item(item, lst=[]):
    lst.append(item)
    return lst
print(add_item(1))
print(add_item(2))
```

Output:

[1]

[1, 2]

Reason:

Default mutable object created only once.

Correct way:

```
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

4. Difference Between Deep Copy and Shallow Copy

Shallow Copy

Copies outer object only.

Nested objects share references.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
b[0][0] = 100
print(a)
```

Output:

```
[[100, 2], [3, 4]]
```

Deep Copy

Copies complete nested structure.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)
```

5. Why Tuple Can Contain Mutable Objects?

Tuple itself is immutable, but contained objects may be mutable.

```
t = ([1, 2], [3, 4])
t[0].append(100)
print(t)
```

Output:

```
([1, 2, 100], [3, 4])
```

Tuple reference immutable, not internal object.

6. Why List is Unhashable?

Hash value must remain constant.

Mutable objects can change, so list cannot be hashable.

```
d = {[1, 2]: "hello"}
```

Output:

```
TypeError
```

7. How Dictionary Handles Hash Collision?

When two keys produce same hash:

1. Python checks equality
2. Finds alternate slot internally

This process is called:

Collision Resolution

8. Time Complexity of Python Data Structures

Operation	List	Set	Dict
Access	O(1)	O(1) avg	O(1) avg
Search	O(n)	O(1) avg	O(1) avg
Insert	O(1) append	O(1) avg	O(1) avg
Delete	O(n)	O(1) avg	O(1) avg

9. Difference Between `sort()` and `sorted()`

<code>sort()</code>	<code>sorted()</code>
Modifies original list	Returns new list
List method	Built-in function

10. What is Interning in Python?

Python reuses immutable objects to optimize memory.

Mostly applies to:

- small integers
 - strings
-

11. Explain Python Memory Management

Python uses:

1. Reference counting
 2. Garbage collector
-

12. What is Reference Counting?

Every object maintains count of references.

When count becomes zero → memory released.

```
a = [1, 2]
b = a
```

Reference count increases.

13. What is Garbage Collection?

Removes cyclic references not cleaned by reference counting.

Uses gc module.

```
import gc
gc.collect()
```

14. Difference Between del, remove, and pop

Method	Purpose
del	Deletes object/reference
remove	Deletes by value

Method	Purpose
pop	Deletes by index and returns value

15. Explain `__slots__`

Used to reduce memory consumption.

```
class Student:
    __slots__ = ['name', 'age']
```

Avoids dynamic `__dict__`.

16. Why Set Does Not Allow Mutable Elements?

Set internally uses hashing.

Mutable objects produce unstable hashes.

Allowed:

```
{1, 2, (3, 4)}
```

Not allowed:

```
{1, 2, [3, 4]}
```

17. Difference Between Iterator and Iterable

Iterable	Iterator
Can be looped	Produces next value
Has <code>__iter__()</code>	Has <code>__next__()</code>

18. What Happens Internally in for Loop?

```
for x in nums:
    print(x)
```

Internally:

```
it = iter(nums)
while True:
    x = next(it)
```

19. Explain Generator

Generator produces values lazily using `yield`.

```
def gen():  
    yield 1  
    yield 2
```

Advantages:

- memory efficient
 - lazy evaluation
-

20. Difference Between Generator and List

Generator	List
Lazy	Stores all values
Less memory	More memory
One iteration mostly	Multiple iteration

21. Explain Python String Internals

Python string:

- immutable
 - unicode-based
 - cached/interned sometimes
-

22. Why Strings are Immutable Internally?

Advantages:

1. Safe hashing
 2. Thread safety
 3. String interning
 4. Optimization
-

23. Difference Between Bytes and String

bytes	str
Binary data	Unicode text
Immutable	Immutable

Example:

```
b = b'hello'  
s = 'hello'
```

24. Explain Unicode in Python

Python 3 strings are Unicode by default.

```
name = "नमस्ते"
```

25. What is Encoding and Decoding?

Encoding

String → bytes

```
s.encode('utf-8')
```

Decoding

Bytes → string

```
b.decode('utf-8')
```

26. Why Dictionary Preserves Insertion Order?

Python 3.7+ maintains insertion order internally.

Older versions did not guarantee this.

27. Explain Dictionary Comprehension

```
squares = {x:x*x for x in range(5)}
```

28. Explain List Comprehension Internally

```
nums = [x*x for x in range(5)]
```

Faster because optimized C-level implementation.

29. Difference Between `append()` and `+=`

```
a.append([3,4])
```

Result:

```
[1,2,[3,4]]
```

```
a += [3,4]
```

Result:

```
[1,2,3,4]
```

30. Explain Python Object Model

Everything in Python is object.

Examples:

- int
- function
- class
- module

All are objects.

31. What is Duck Typing?

Python focuses on behavior instead of type.

```
if object.quack():
```

"If it behaves like duck, treat it as duck."

32. What is EAFP in Python?

EAFP stands for:

Easier to Ask Forgiveness than Permission

It is a programming style commonly used in Python where:

- You try to perform an operation directly
- If an error happens, handle it using `try-except`

Instead of checking everything beforehand.

Basic Idea

Instead of:

```
if key in data:  
    print(data[key])
```

Pythonic EAFP Style:

```
try:  
    print(data[key])  
except KeyError:  
    print("Key not found")
```

Why Python Prefers EAFP

Python assumes:

"Operations usually succeed"

So instead of many condition checks:

- perform operation directly
- handle exceptions only if needed

This makes code:

- cleaner
- faster sometimes
- more readable

33 Difference Between `*args` and `**kwargs`

`*args`

`**kwargs`

Positional arguments Keyword arguments

34. Explain Packing and Unpacking

```
a,b = [1,2]
```

35. What is Memory Leak in Python?

Objects remain referenced unintentionally and are not garbage collected.

36. Explain Circular Reference

```
a = {}  
b = {}  
a['b'] = b  
b['a'] = a
```

Handled by garbage collector.

37. Difference Between Heap and Stack Memory

Stack	Heap
Function calls	Objects
Automatic	Dynamically allocated

38. What is Intern Pool?

Python stores reusable immutable objects in memory pool.

39. Explain `id( )` Function

Returns memory identity.

```
x = 10  
print(id(x))
```